MultiLayer Compressed Counting Bloom Filters

Conference Paper *in* Proceedings - IEEE INFOCOM · May 2008

DOI: 10.1109/INFOCOM.2008.71 · Source: IEEE Xplore

CITATIONS
57

READS
631

4 authors:

Domenico Ficara
University of Pisa
56 PUBLICATIONS **514** CITATIONS
SEE PROFILE

Stefano Giordano
University of Pisa
307 PUBLICATIONS **3,120** CITATIONS
SEE PROFILE

Gregorio Procissi
University of Pisa
127 PUBLICATIONS **1,409** CITATIONS
SEE PROFILE

Fabio Vitucci
University of Pisa
46 PUBLICATIONS **487** CITATIONS
SEE PROFILE

This full text paper was peer reviewed at the direction of IEEE Communications Society subject matter experts for publication in the IEEE INFOCOM 2008 proceedings.

MultiLayer Compressed Counting Bloom Filters

Domenico Ficara, Stefano Giordano, Gregorio Procissi, Fabio Vitucci
Dept. of Information Engineering, University of Pisa, ITALY
Email: {domenico.ficara, stefano.giordano, gregorio.procissi, fabio.vitucci}@iet.unipi.it

***Abstract*—Bloom Filters are efficient randomized data structures for membership queries on a set with a certain known false positive probability. Counting Bloom Filters (CBFs) allow the same operation on dynamic sets that can be updated via insertions and deletions with larger memory requirements.**

This paper first presents a new upper bound for counters overflow probability in CBFs. This bound is much tighter than that usually adopted in literature and it allows for designing more efficient CBFs. Three novel data structures are proposed, which introduce the idea of a hierarchical structure as well as the use of Huffman code. Our algorithms improve standard CBFs in terms of fast access and limited memory consumption (up to 50% of memory saving): the target could be the implementation of the compressed data structures in the small (but fast) local memory or "on-chip SRAM" of devices such as Network Processors.

***Index Terms*—Counting Bloom Filter, Huffman coding, Network Processor, Hashing functions, MultiLayer structure**

I. Introduction

Nowadays streamed data processing is a basic problem in many areas related to computer applications. In particular, detecting whether an item belongs to a set is one of the most challenging tasks, especially when the amount of data to be processed per unit of time is very large and rapidly changes.

A Bloom Filter (BF) is a simple data structure for information representation and query processing. It is a randomized method based on hash functions; thus, it allows for false positives, but the space savings often outweigh this drawback. BFs were introduced by Burton Bloom [1] in the 1970s for database applications, but recently they have received a great attention also in the networking area [2], for collaborating in overlay and peer-to-peer networks, packet routing, and measurements. BFs are also proposed for many distributed networking protocols: for example, in order to share web cache, a proxy periodically broadcasts BFs that represent the contents of their cache. In this situations, BFs are not only data structures but also messages being transmitted in a network. Thus, several performance parameters have to be taken into account: the probability of false positives, memory size, number of items to be managed and transmission size.

BFs do not address the issues of inserting and deleting items in the set. For example, a set may change over time, with elements being inserted and deleted. Deletion cannot be done by simply changing ones into zeros, as a single bit may correspond to multiple elements. In order to allow these operations, Counting Bloom Filters (CBFs) have been designed [3]. They are based on the same idea of BFs, but they use fixed size counters (also called bins) instead of single bits of presence. When an item is inserted, the corresponding counters are incremented; deletions can now be safely done by decrementing the counters. CBFs present the problem of counters overflow, which has to be considered in the design.

This paper presents a new upper bound for overflow probability in CBFs. This bound is much tighter than that commonly utilized in literature and it is useful for our design of efficient solutions. Thus, three new data structures are proposed, which take advantage of the bound and improve CBFs in terms of fast access and limited memory consumption (up to 50% of memory saving in comparison with the standard solutions). The target could be the implementation of the compressed data structures in the small but fast local memory or "on-chip SRAM" of devices such as Network Processors (NPs).

Moreover, our proposed algorithms introduce the idea of layer hierarchy in the CBF data structure, which allows to take advantage of the built-in memory hierarchy of many systems.

Therefore, the main contributions of this work are:

- a tighter upper bound for overflow probability in CBFs (exploited in the design of high performance solutions);
- the use of Huffman code in CBF, which is optimal for independent symbols (such as the bins of a CBF);
- the idea of a hierarchical multilayer structure;
- the proposal of an efficient CBF for systems with limited memory such as NPs and programmable routers.

The next section presents the most important works on CBFs. In section III the theoretical results at the basis of our research are shown. Sections IV, V, and VI describe the methods proposed in this paper, in terms of algorithms, properties and simulation results. The first solution introduces a multilayer structure, the second one the Huffman coding and the third one combines both the previous ideas into the MLCC-BF data structure. Finally, a comparison among our algorithms and the algorithms defined in literature is performed, by adopting Intel IXP2800 as a referential hardware platform. Section VIII gives the conclusions of our work.

II. Background on Bloom Filters

A Bloom Filter represents a set S of n elements from a universe U by using an array of m bits, denoted by $B[1], ..., B[m]$, initially all set to 0. The filter uses k independent hash functions $h_1, ..., h_k$ with $\log_2(m)$ bits long output, that independently map each element in the universe to a random number uniformly distributed over the range. For each element x in S, the bits $B[h_i(x)]$ are set to 1, for $1 \leq i \leq k$ (a bit can be set to 1 multiple times).

To answer a query of the form *"Is y in S?"*, we check whether all $B[h_i(y)]$ are set to 1. If not, y is not a member of

978-1-4244-2026-1/08/$25.00

S, by construction. If all $B[h_i(y)]$ are set to 1, it is assumed that y is in S, hence a BF may yield a false positive. The probability of a false positive f can be tuned by choosing the proper values for m and k. It is a well-known result [3] that the minimum f is obtained for $k = (m/n)\ln 2$. In this configuration, all bits $B[1], ..., B[m]$ are set or cleared with probability $p = 1/2$ (thus, roughly, the same number of ones and zeros are present in the BF).

Many works about BFs have been presented, and the major improvements are compressed BFs [4], distance-sensitive BFs [5], dynamic BFs [6], space-code BFs [7].

As previously stated, BFs do not allow insertion and deletion of an item in the set. Therefore, CBFs have been introduced, which use m fixed size bins instead of m single bits of presence. When an item is inserted (or deleted), the corresponding counters are incremented (or decremented).

However, CBFs present the problem of counters overflow, which has to be considered in the design. Although for most network applications four bits long counters are sufficient [2], the distribution of counters load across bins changes dramatically (according to Poisson arrivals [3]), suggesting that four bits per bin is a safe choice and that a certain amount of compression is achievable. Moreover, by using a fixed number of bits, the problem of counters overflow in CBFs is not completely solved. It results in a lack of adaptiveness and inaccuracy of stored information.

In order to waive these limitations and achieve better performance, many improvements to CBFs have been done. Mitzenmacher [4] shows that unbalancing the number of ones and zeros in a standard BF can help achieving a good compression ratio before transmission (e.g. for web-caching application). This way, by keeping the same amount of bits of the uncompressed case, it is possible to either reduce the false positive probability or use a lower number of hash functions.

Spectral Bloom Filters (SBFs) [8] are an extension of standard BFs to multi-sets, allowing estimates of the multiplicities of individual items with a small error probabilities. The word "Spectral" means that SBFs allow only filtering of elements whose multiplicities are within a requested spectrum (therefore they do not preserve bins from overflow in a conclusive way). The main goal of SBFs is the optimal counter space allocation, so they dynamically vary the size of their counters in order to minimize the number of necessary bits. To achieve this flexibility, SBFs include additional slack bits among the counters and complex index structures, that increase both memory needs and access time as compared to standard CBFs. Finally, SBFs introduce techniques for filter compression based on Elias code, that reduce the transmission size of data structures but increase again the processing load.

Dynamic Count Filters (DCFs) [9] are data structures designed for speed and adaptiveness in a very simple way. They do not require the use of indexes, thus obtaining a fast access time, and avoid permanently counters overflow. DCFs consist of two different vectors: the first one is a basic CBF with counters of fixed size, the second one is the Overflow Counter Vector, which has a counter for each element of first vector that keeps track of the number of overflow events. The size of counters in Overflow Counter Vector changes dynamically to avoid saturation; this implies that, for each update, a structure rebuilding is required. Moreover, the decision of having the same size for all these counters (for direct access) entails that many bits are not used. Therefore, this solution can be improved, especially in terms of memory consumption.

The d-left CBFs (dlCBFs) [10] are simple alternatives based on d-left hashing and fingerprints of bins. They do not rely on the principles of Bloom Filters, but they offer the same functionalities. The dlCBFs use less space, generally saving a factor of two or more for the same fraction of false positives, and the construction is very simple and practical, much like the original Bloom Filter construction. Indeed the simplicity in constructing and maintaining data structures is maybe the greatest contribution of [10] as compared to previous works. Moreover, even dlCBFs have the limitation of potential counters overflow and the need for an additional fingerprint for each bin in the data structure.

The memory utilization is the parameter that is better taken into account in this work. As previously mentioned, there are several cases where network bandwidth is still expensive and transmission size becomes a fundamental parameter (e.g., Web cache sharing or P2P applications). Moreover, although memory appears plentiful today, there are many hardware architectures used in network devices (e.g. Network Processors) that may take advantage of using very space-efficient data structures, in terms of both performance and costs. Indeed, memory saving can greatly speedup a device by requiring rare access to slower off-chip memory; further, while ordinarily DRAM memory is cheap, fast SRAM memory and especially on chip SRAM continue to be comparatively scarce. All these issues have led our research, which had the target of an efficient and practical data structure for CBF.

III. Theoretical Results

In this section we present the main theoretical results on the CBF counter overflow probability and on Huffman coding of bin counters that will be the basis of the data structures proposed in the rest of the paper.

The following classical result [2] on CBF gives a bound on the overflow probability $P(\varphi \geq j)$ that is widely adopted to design the bin size:

$$P(\varphi \geq j) \leq \left(\frac{enk}{jm}\right)^j \tag{1}$$

However, (1) is pretty loose; the next theorem 1 presents a tighter bound for $P(\varphi \geq j)$.

Lemma 1: Let φ be a CBF counter value and $\alpha = \frac{nk}{m-1}$. If $\alpha < 1$, the function $\chi(j) = P(\varphi = j)$ is a monotonically decreasing function.

Proof: The probability of the event $\{\varphi = j\}$, for $j \geq 1$, is given [2] by:

$$\chi(j) = \binom{nk}{j}\left(\frac{1}{m}\right)^j\left(1 - \frac{1}{m}\right)^{nk-j}$$

The ratio between two consecutive values is:

$$\frac{\chi(j+1)}{\chi(j)} = \frac{nk-j}{j+1}\frac{1}{m-1} < \frac{\alpha}{j+1} < 1 \qquad (2)$$

which gives the proof. ■

For $k = (m/n)\ln 2$, $\alpha = (m \times \ln 2)/(m-1)$. α is less than 1 for $m > (1-\ln 2)^{-1} \approx 3.26$. In the CBFs, the previous condition is always satisfied, since $m \gg 1$.

Theorem 1: Let φ be a CBF counter value and $\alpha = \frac{nk}{m-1}$. If the number of hash functions is chosen so as to minimize the probability f of false positive (i.e., $k = (m/n)\ln 2$), then:

$$P(\varphi \geq j) < \frac{\alpha(j+1)}{j(j+1-\alpha)} P(\varphi = j-1)$$

Proof: By repeadetly applying eq. 2:

$$P(\varphi \geq j) = \sum_{i=j}^{+\infty} P(\varphi = i) < P(\varphi = j) \sum_{i=0}^{+\infty} \frac{j!\alpha^i}{(j+i)!} \qquad (3)$$

The right hand sum of (3) can be bounded as:

$$\sum_{i=0}^{+\infty} \frac{j!\alpha^i}{(j+i)!} < \sum_{i=0}^{+\infty} \left(\frac{\alpha}{j+1}\right)^i = \frac{j+1}{j+1-\alpha}$$

to finally obtain:

$$P(\varphi \geq j) < \frac{\alpha(j+1)}{j(j+1-\alpha)} P(\varphi = j-1) \qquad (4)$$

■

Corollary 1: Under the previous results, if $\alpha < 1$:

$$P(\varphi > j) < P(\varphi = j-1) \qquad (5)$$

Proof: From (3), by changing the lower limit of the series from 0 to 1, we obtain:

$$P(\varphi > j) < \frac{\alpha^2}{j(j+1-\alpha)} P(\varphi = j-1) \qquad (6)$$

Then, considering that $j \geq 1$, $\alpha^2/(j(j+1-\alpha)) < 1$. ■

Lemma 1 allows to approximate $P(\varphi = 0)$ and $\mathbb{E}[\varphi]$:

$$1 = \sum_{j=0}^{+\infty} P(\varphi = j) \simeq P(\varphi = 0) \sum_{j=0}^{\infty} \frac{\alpha^j}{j!} = P(\varphi = 0) e^{\alpha} \qquad (7)$$

Then $P(\varphi = 0) \simeq e^{-\alpha}$. As for the expectation of φ we get:

$$\mathbb{E}[\varphi] = \sum_{j=0}^{+\infty} jP(\varphi = j) \simeq P(\varphi = 0) \sum_{j=0}^{\infty} j\frac{\alpha^j}{j!} = \alpha \qquad (8)$$

If the CBF minimizes f, $\mathbb{E}[\varphi] \simeq \ln 2 = 0.693$, which is a a very tight approximation in several cases.

It is interesting to see that, as shown in fig. 1, the previous bound can be much tighter than the widely used (1). For instance, if $n = 1000$, $k = 10$ and $m = nk/\ln 2$, eq. (1) yields $P(j > 15) \leq 1.37 \times 10^{-15}$ while our bound produces $P(j > 15) < 1.51 \times 10^{-16}$, with a gain of an order of magnitude. Moreover, the results of this first theorem are the basis for the following one.

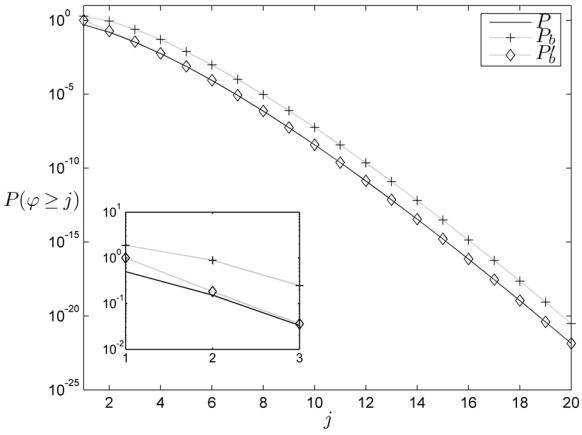

Figure 1. Bounds comparison for $n = 1000$, $k = 10$ and $m = nk/\ln 2$. P is the actual $P(\varphi \geq j)$, P_b is the well known (1) while P_b' is that provided by the theorem 1. In the smaller graph, a zoom on the contour of $j = 2$. P_b' is always tighter than P_b.

Theorem 2: Let $H(\sigma)$ be the Huffman coding of σ, $len(\cdot)$ the "bit-length" operator, φ a CBF counter value; then:

$$len(H(\varphi)) = \varphi + 1$$

Proof: Huffman codes can be obtained by using a binary tree. The tree is constructed from a list of N nodes (symbols) whose weights correspond to the symbol probabilities.

The whole procedure is the following:

- let x and y be the two nodes with the lowest weight;
- x and y are aggregated into a parent node whose weight is set to the sum of the two nodes;
- the parent node replaces x and y in the list.

These steps are repeated until the list contains one node only.

To perform Huffman coding of CBF bin counters, we first construct a tree whose nodes $X_0, ..., X_N$ correspond to the possible values of the counters $j = 0, ..., N$; the weight of the j-th node is set to $P(\varphi = j)$. Let L_τ be the list of nodes at step τ and let X^τ be the parent node to be created at this step. Suppose we have $L_\tau = \{X_0, X_1, \ldots, X_{N-\tau-1}, X^{\tau-1}\}$; the weight of the parent node $X^{\tau-1}$ created at the previous step is $P(X^{\tau-1}) = P(j > N-\tau-1)$. By corollary 1:

$$P(X^{\tau-1}) < P(j = N-\tau-2)$$

Moreover, the previous inequality also implies that $P(X^{\tau-1})$ is smaller than any of the values in the set $\{P(X_0), \ldots, P(X_{N-\tau-2})\}$. Then, at step τ, the nodes with the smallest weights are $X^{\tau-1}$ and $X_{N-\tau-1}$ and they shall be aggregated into the parent node X^τ. Thus:

$$L_\tau = \{X_0, ..., X_{N-\tau-2}, X_{N-\tau-1}, X^{\tau-1}\} \Rightarrow$$
$$\Rightarrow L_{\tau+1} = \{X_0, ..., X_{N-\tau-2}, X^\tau\}$$

The resulting tree turns out to be completely unbalanced (i.e., the depth of all N nodes is given by the sequence of the first N naturals) such as the one of fig. 2. Therefore, the depth of node φ is $\varphi + 1$, i.e. the encoding of the value φ is $\varphi + 1$ bit-long. ■

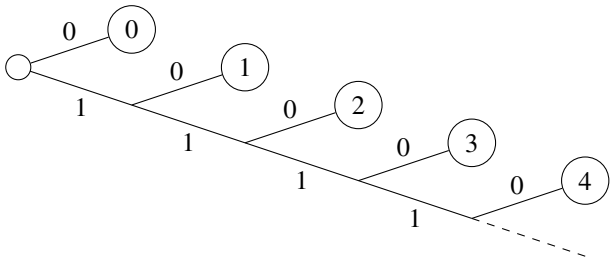

Figure 2. A Huffman tree for the CBF bin counters.

IV. MULTILAYER HASHED CBF (ML-HCBF)

A. The algorithm

The basic idea of the first data structure proposed in this paper is the addition, to the usual CBF vector (CBFV), of a stack of hash-based vectors (HBVs). To the best of our knowledge, although a limited degree of hierarchy is sometimes obtained by adding a CAM [11] or another counter [9], this is the first attempt to introduce the idea of a hierarchy of arrays in CBFs, which results in a *MultiLayer* structure where counters may span over different levels.

The target of our data structure is an improvement of CBFs by avoiding counters overflow and reducing memory needs. The drawback, as we will see below, is a very slight increase of complexity for the insertion/deletion of an element. Instead, the probability f of false positives does not vary, because it depends on the first CBF vector only.

Let x_0 and m_0 be the bit-size and the number of bins in the CBFV respectively, while $x_1 \dots x_N$ and $m_1 \dots m_N$ (all of them powers of 2) represent the bit-size and number of bins for the N vectors $HBV_1, \dots, HBV_N$. These values are set to obtain a low probability of counters overflow. Basically, we define a counter value as the sum of its CBFV element and its potential corresponding elements in the HBV stack.

A set of $k + N$ hash functions h_i are needed. The first k functions (that address the CBFV) have an output length of $\log_2(m_0)$ bits while the other N ones (one for each HBV) provide $\log_2(m_i)$ bits (where $i = 1, \dots, N$) and must be perfect (and minimal, for memory requirements). The need for minimal perfect hash functions (MPHFs) increases the operational complexity whenever an element has to be inserted or deleted from the sets involved in the perfect hashing (i.e., if either an insertion causes a bin saturation or a deletion decrements a bin value to 0). These events are rare, therefore the construction cost of the required MPHFs is minimal.

When an operation of either inserting or deleting an element s is required, the current counter Cc_i (with $i = 1, \dots, k$) for each of the first k hash functions must be found. Therefore, at first $CBFV[h_i(s)]$ has to be considered: if it is already saturated (i.e. it has reached $2^{x_0} - 1$), the counter position is hashed. The obtained value is used to address a counter in HBV_1. If its value is also saturated, the same procedure is repeated: the position in HBV_j is hashed to obtain the index for HBV_{j+1} until a non-saturated counter (Cc_i) is found. Hence, the actual value of the counter is the sum of $CBFV[h_i(s)]$ and all the explored HBV_j elements. The overall process to find the current counters is shown in the pseudocode 1 and in the simple example of fig. 3.

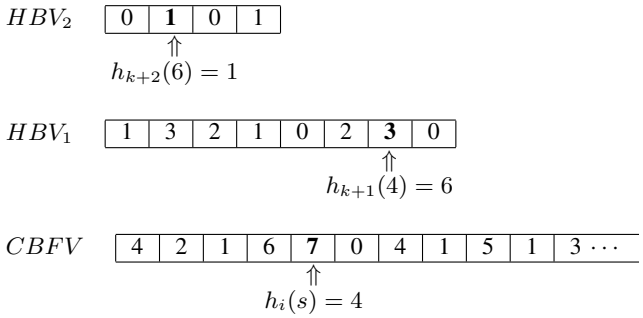

Figure 3. Process of finding a current counter for ML-HCBF (with $x_0 = 3$ bits and $x_1 = x_2 = 2$ bits). The hash function h_i gives 4 as outcome: $CBFV[4]$ is saturated, therefore, counter position (4) is hashed and the output is 6. Also $HBV_1[6]$ is saturated, then its position is hashed: the outcome is 1. $HBV_2[1]$ is the current counter Cc_i for s. Its actual value is $1+3+7 = 11$.

Algorithm 1 Pseudocode for finding current counters

```
1:  for i ← 1, k do
2:      Cc_i ← CBFV[h_i(s)]
3:      if CBFV[h_i(s)] = 2^{x_0} − 1 then
4:          j ← 1
5:          pos ← H_j[h_i(s)]
6:          repeat
7:              Cc_i ← HBV_j[pos]
8:              pos ← H_j[pos]
9:              j ← j + 1
10:         until HBV_j[pos] ≠ 2^{x_j} − 1
11:     end if
12: end for
```

Each current counter Cc_i (where $i = 1, \dots, k$) is incremented by one for inserting an element. Instead, if an element must be deleted, Cc_i is decremented; if it is already equal to 0, the relative counter of previous layer is decremented.

To perform a membership query for an element s, we check whether all $CBFV[h_i(s)]$ are non-zero values. It is exactly the same simple procedure as for standard CBF.

B. Properties

If the number of HBVs is N, the number of bins in CBFV is m_0 (set for minimizing f) and in HBV_j is $m_j = \alpha_j m_0$ (where α_j is a negative power of 2, as hashing reduces the range of values for each layer):

- the maximum supported value for a counter is

$$\varphi_{max} = \sum_{j=0}^{N} 2^{x_j} - N - 1$$

- the average size of the overall data structure is

$$\mathbb{E}[S] = m_0 \left(x_0 + \sum_{j=1}^{N} x_j \alpha_j \right)$$

where $\alpha_j = 2^{\lceil \log_2 P(\varphi > \varphi_{j-1}) \rceil}$.

C. Operational Complexity

As above mentioned, a membership query for an element s is performed the same way as in standard CBFs: we check whether all $CBFV[h_i(s)]$ (where $i = 1, \ldots, k$) are non-zeros. Therefore k hash operations are required and lookup complexity is $O(1)$.

The complexity for insertion and deletion of an element is almost the same. We use k hash functions for the CBFV and one for each HBV we reach, thus resulting in the following average number ω of hash operations in the ordinary case:

$$\omega = k \left(1 + \sum_{j=1}^{N} P(\varphi > \varphi_j) \right)$$

Finally, an operation of incrementing/decrementing a counter has to be made. The cost of insertions/deletions increases because of the complexity of the minimal perfect hashing scheme that must cope with a change in the set of elements to be "perfectly" hashed. Such a complexity depends on the cardinality of the set, which is the number of saturated bins at the current counter's level (m_{j+1} at layer j). Moreover, in order to compute the average complexity, the cost must be weighted with the probability of a transition toward a saturated bin. Therefore, even if the cost of the perfect hashing [12] at layer j is $O(m_{j+1})$, in the average case the increment is limited, as we observed during simulations.

To sum up, the total amount of operations is very close to k, provided that the structure is designed to have low overflow probabilities (see tab. I). Indeed, the higher the counter, the lower the MPHF's added complexity (the fewer the elements to be hashed); on the other hand, the lower the counter, the fewer the layers, the jumps and the memory accesses. These two effects compensate each other, therefore the number of operations in the worst case is constant and limited.

If the data structure is well designed (i.e. low overflow probability), it is worth observing that the more the number of levels, the larger φ_{max}, thus the smaller the overall overflow probability, with almost the same number of operations.

D. Simulation results

In this section, in order to better understand how the parameters affect data structure behavior, simulation results are shown about the memory usage of ML-HCBF in comparison with standard CBF. In the simulation runs, a set of 2000 elements has been generated to test the different data structures. For CBF and for the first layer of ML-HBCF, 10 hash functions are used and the number of bins m_0 is selected so as to minimize the probability of false positives (according to [3]).

For each run (i.e. an entry of tab. I), we have set a configuration for ML-HCBF (number of layers and size of bins per layer) and calculated the overflow probability. Then, by theorem 1, we have found the number of bits for bin required in a standard CBF to obtain the same overflow probability. Therefore, the results in tab. I refer to structures with the same probabilities of overflow and false positives: memory saving of ML-HCBF in comparison with standard CBF is clear.

Table I
DATA STRUCTURES COMPARISON. SIZE IS EXPRESSED IN KBYTES

$(x_0, \ldots, x_N)$	$P(\varphi > \varphi_{max})$	$ML-HCBF$	CBF	$ratio$
$(2,2,2)$	5×10^{-8}	7.55	11.16	0.67
$(2,2,2,2)$	1.28×10^{-11}	7.55	12.6	0.59
$(2,2,2,2,2,2)$	1.1×10^{-19}	7.55	14.68	0.51
$(2,2,3,3)$	1.34×10^{-22}	7.55	15.22	0.50
$(2,2,4)$	4.4×10^{-24}	7.55	15.47	0.49
$(2,3,3)$	2.7×10^{-18}	7.6	14.4	0.53

Finally, notice that the size of ML-HBCF depends almost exclusively on the number of bits of the first level. The best memory saving are obtained by using 2 bits for CBFV bins. Further, as shown by the first entries in the table, by increasing the number of levels in ML-HCBF, memory consumption is nearly the same but the overflow probability decreases and the gain with respect to CBF increases.

V. HUFFMAN SPECTRAL BLOOM FILTERS

In order to introduce our second method, the Huffman Spectral Bloom Filter (HSBF), we begin by recalling Spectral Bloom Filters [8]. They use a memory-efficient structure that encodes any bin with Elias coding. This way, bins do not have a fixed position and, for all k hash functions, we have to find the right bin it points to by looking up a certain amount of words. Lookup implies to decode a number of bins until the right one is found. Moreover each insertion and deletion imply a potential shift of the whole structure.

To simplify these operations, SBFs divide the entire structure in subsegments and use a set of tables in aid to the lookup. In addition, a certain number ε of empty bits (called slack bits) are inserted to reduce shifts operations for insertions and deletions. Elias compression scheme is a perfect choice when dealing with large numbers, such as those of multiset membership query applications. However, for smaller values (recall that in a regular CBF, 16 is widely considered as a high loose bound), other codings can perform better.

Our proposal is to encode a number σ with σ consecutive ones and a trailing zero (fig. 2). This way, the encoding produces $\sigma + 1$ bits for symbol σ: it is a Huffman coding, as shown in theorem 2. This is a major advantage since Huffman is the minimum redundancy coding for independent symbols such as the bins of a CBF.

Moreover, our coding scheme allows an easy lookup since most processors provide an instruction that counts the number of bits set to one in a word (*popcount*). By taking advantage of such an instruction, we do not have to decode each value we find during lookup, but simply count the number of cleared bits in a word. The number of cleared bits is the number of symbols encoded in that word (see example in fig. 4). Clearly, we still have to perform a shift for each insertion or deletion and we need a table to speed up lookup but the total size of the structure is very close to the minimum (given by the entropy of all symbols).

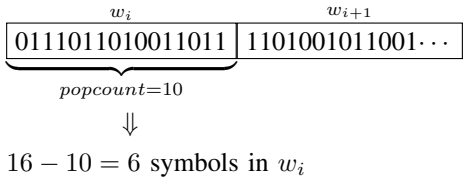

Figure 4. Example of fast lookup through popcount.

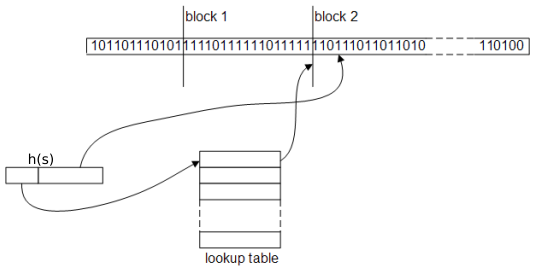

Figure 5. An example of HSBF.

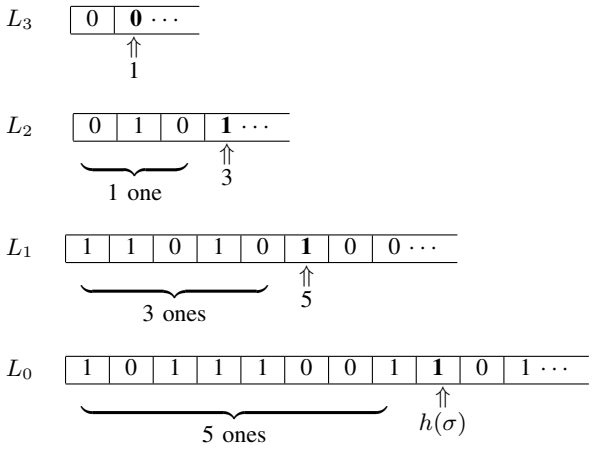

Figure 6. ML-CCBF example. The resulting Huffman code for φ is 1110.

A. Size

We divide the bins in B blocks of D bins and we address the blocks with the table. The average size of the HSBF is:

$$\mathbb{E}[S] = m(1 + \mathbb{E}[\varphi]) + B\left(\varepsilon + \log_2\left[(m - D)\left(\varphi_{max} + 1\right)\right]\right)$$

where ε is the number of slack bits kept at the end of each block. The last part of the above formula takes into account the table size. The table is addressed by the first $\log_2 B$ bits of the hash, the remaining bits represent the bin index. Each entry of the table represents the starting address of the corresponding block thus requiring less than $(m - D)(\varphi_{max} + 1)$ bit.

B. Lookup

As for operation complexity, a lookup requires k hash functions and, for each of them, a check in the table and a search in the corresponding block for the bin we need (see fig. 5). Thus, on average, $D/2$ bins have to be looked and $W/(\mathbb{E}[\varphi] + 1)$ bins will be found in a word of W bits. The overall average number of operations for a lookup is then:

$$\omega = k\left(\frac{D(\mathbb{E}[\varphi] + 1)}{2W}\right)$$

As shown in section III, $\mathbb{E}[\varphi] \simeq \ln 2$. Therefore, the average number of operations for a lookup is constant and its complexity is $O(1)$.

C. Insertion/Deletion

In order to insert a new element, we need to perform a lookup and to add a "1" digit for each bin in the code. This corresponds to shifting all the bits at the bin's right by one position and a table update. Thus, for all insertions, the number of operations is:

$$\omega = k\left(\frac{D(\mathbb{E}[\varphi] + 1)}{W}\right)$$

It is straightforward to see that, since even deletion requires a lookup and a shift, the overall cost is the same as insertion. The complexity of these operations is $O(1)$, as for lookup.

VI. MultiLayer Compressed CBF

The drawbacks of the algorithm and data structure described in sec.V, as well as SBF, are related to the memory wastage due to slack bits and to the complexity of a searching based lookup (even if aided by index tables).

In the following, the MultiLayer Compressed Counting Bloom Filter (ML-CCBF) is presented, which is a CBF that reduces the memory requirements and the complexity of lookup. The idea is, again, to explode the CBF along another dimension, hence creating a *multilayer* structure. This construction, in conjunction with the Huffman coding defined in sec.V, provides a stack of bitmaps $(L_0, ..., L_N)$, where the first layer L_0 is a standard BF. The other layers are built and modified dynamically when needed.

Let *popcount(u)* be the number of 1s in the bitmap $(0, ..., u - 1)$; the construction is as follows:

- L_i keeps all the i-th binary digits of our Huffman encoded counters;
- on L_i, the j-th bit belongs to the counter whose *popcount* on L_{i-1} is j.

Figure 6 shows an example of a ML-CCBF. In the example, we are counting a bin φ for symbol σ. The bin at layer 0 is pointed by the hash function $h(\sigma)$. The number of ones before $h(\sigma)$ is computed (i.e. $popcount(h(\sigma)) = 5$) and used as index for layer 1. The procedure is repeated until we find a "0" digit (that is the end of the code). Therefore the resulting Huffman code for the counter is 1110, which corresponds to value 3.

A. Complexity and properties

One of the most significant advantage of our algorithm is that it is an extension of a standard BF. Thus, the lookup is as simple and fast as in a standard BF as we need to check only bits at layer 0. Therefore the lookup complexity is $O(1)$.

Instead, for insertions and deletions, we need to explore different layers in the structure. We refer to m_i as the number

of bits in layer i. The size of layer i can be obtained as:

$$m_i = m_0 P(\varphi \geq i)$$

Since jumping one layer up requires a *popcount* on a potentially large number of bits, we divide all layers in blocks of the same bit-size D and add a table for each level. When computing $popcount(u_j)$ at layer j, the first $\log_2(m_j/D)$ bits of u_j are used as index to table j. Each entry of the table represents the number of ones preceding the start of the block. Thus, if W is the number of bits in a word, the actual *popcount* operation works only on less than D/W words. Therefore, the average cost of a *popcount* is $1 + \frac{D}{2W}$.

Algorithms 2 and 3 show the pseudocode for insertion and deletion procedures in a ML-CCBF. Both operations require, for all k bins, the complete lookup of multiplicity (by exploring a certain amount of layers), a shift by one position and the update of the last explored table. Such an update simply consists of an increment or a decrement on a limited number of entries. Therefore, the average number of operations for insertion and deletion is given by:

$$\omega = k \left[\mathbb{E}[\varphi] \left(1 + \frac{D}{2W} \right) + 2 \right]$$

Once again, $\mathbb{E}[\varphi] \simeq \ln 2$, thus the average amount of operations is fixed and the complexity for insertion/deletion is $O(1)$.

Algorithm 2 The insertion of an element in a ML-CCBF

```
1: for i ← 1, k do
2:     j ← 0
3:     u_0 ← h_i(s)
4:     while (L_j(u_j) = 1) do
5:         u_{j+1} ← popcount(u_j)
6:         j ← j + 1
7:     end while
8:     L_j(u_j) ← 1
9:     u_{j+1} ← popcount(u_j)
10:    j ← j + 1
11:    L_j(u_j + 1, ..., m_j + 1) ← L_j(u_j, ..., m_j)
12:    m_j ← m_j + 1
13:    L_j(u_j) ← 0
14:    UpdateTable(L_j)
15: end for
```

Algorithm 3 The deletion of an element in a ML-CCBF

```
1: for i ← 1, k do
2:     j ← 0
3:     u_0 ← h_i(s)
4:     while do(L_j(u_j) = 1)
5:         u_{j+1} ← popcount(u_j)
6:         j ← j + 1
7:     end while
8:     L_j(u_j, ..., m_j) ← L_j(u_j + 1, ..., m_j + 1)
9:     m_j ← m_j − 1
10:    L_{j−1}(u_{j−1}) ← 0
11:    UpdateTable(L_j)
12: end for
```

B. Size

ML-CCBF is a multilayer transposition of the algorithm shown in sec.V, with no need for slack bits. Hence, it results in a lower memory requirement:

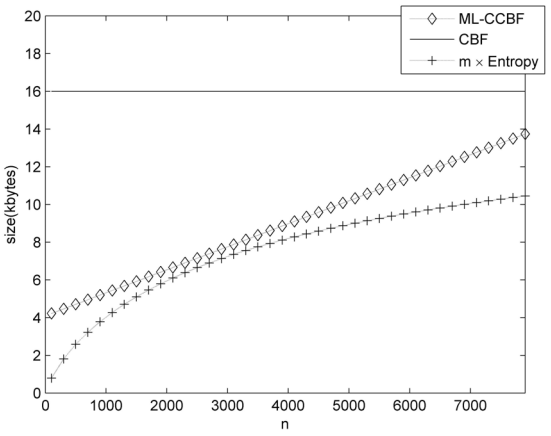

Figure 7. Size comparison among ML-CCBF, CBF and $m \times$ Entropy.

$$S = m_0 + \sum_{i=1}^{m_0} \varphi_i + \sum_{i=1}^{n_{tab}} TS_i$$

TS_i is the size of the table required for layer i, which needs $n_i = \lceil m_i/D \rceil$ entries of size $\log_2(m_i)$, thus resulting in:

$$TS_i = n_i \log_2(m_i) = \left\lceil \frac{m_0}{D} \right\rceil P(\varphi \geq i) \log_2 \left[m_0 P(\varphi \geq i) \right]$$

The average amount of required memory is then:

$$\mathbb{E}[S] = m_0(1 + \mathbb{E}[\varphi]) + TS$$

A closed form expression for $TS = \sum_{i=1}^{n_{tab}} TS_i$ is not simple to obtain in a general case. However, we use the results of theorem 1 (see the Appendix) to compute a bound for TS.

If $\alpha = \ln 2$ to minimize the false positive probability, then:

$$TS \leq \left\lceil \frac{m_0}{D} \right\rceil (2 \log_2(m_0) - 1.85)$$

Figure 7 shows the comparison among the sizes of ML-CCBF, standard CBF and the minimum amount of bits for independent symbols (BF entropy $= m \times$ entropy), for $k = 10$ and $m = 32768$. The memory saving of our method is clear as it approaches the minimum value. Note that the optimal number of elements $n = 2270$ that minimizes f, minimizes the distance from the BF entropy as well.

VII. Comparative Analysis

For the evaluation of the algorithms proposed in this paper and the comparison with other known in literature, the Network Processor Intel IXP2800 has been taken as referential hardware architecture. NPs are platforms that offer very high packet processing capabilities (e.g. for gigabit networks) and combine the programmability of general-purpose processors with the high performance typical of hardware-based solutions. The IXP2800 is designed to perform a wide range of functionalities, including multi-service switches, routers, and broadband access devices. It is a fully programmable network processor, characterized by a hierarchy of processing units (a XScale core and 16 32-bit microengines MEv2) and memory

devices (4KB of local memory, 16KB of scratchpad memory, besides external memories of the host card). The bigger the memory, the slower the access to it. For more details about Intel IXP2800 we refer to [13].

The hierarchy of memory devices in the IXP2800 reflects the memory architecture of many systems, which present small fast memories and slower big ones. Therefore, although referred to a certain hardware platform, the results of our research are very general.

As shown in tab. II, we have weighted the operations of the algorithms in terms of clock cycles for microengines. Microengines are just the processors designed to handle fast data path; the operations weights are set according to the IXP2800 Hardware Reference Manual [14]. For the construction of a MPHF, we have simulated the algorithm in [12] and measured its cost.

Each algorithm has been simulated and its performance has been measured in terms of memory consumption and processing load for lookup and insertion/deletion. In simulation runs, the total number of data elements is $n = 2000$, $k = 10$, and the number of bins for the main vector is 2.8×10^4, thus minimizing the probability of false positives. For the algorithms which divide data structure in subsegments, the number of blocks is $B = 64$. All other parameters are set to obtain about the same probability of false positives among the different algorithms and to be able to manage the same number n of elements. Moreover, for the algorithms which present a hierarchical structure, we have located each substructure in the fastest memory as possible (see tab. III).

In ML-HCBF, we have set a configuration of four layers $(2, 2, 3, 3)$. The first vector $CBFV$ has been stored in scratchpad memory, thus a lookup requires accesses to scratchpad. Vectors HBV_1, HBV_2 and HBV_3 can be located in local memory, thus an insertion/deletion needs also a certain number of accesses to this memory. However, in this layer configuration the probability of overflowing $CBFV$ is very low (0.033), thus accesses to local memory are not frequent.

Concerning ML-CCBF, the main BF vector L_0 and index tables are stored in local memory, while the remaining vectors in scratchpad. A lookup only requires checking the first vector, therefore only local memory is accessed. For insertion and deletion we still need to explore different layers in the structure, thus both memories are accessed.

For a standard CBF, built with four bits for bin, the overall structure has been located in scratchpad. Therefore lookup, insertion and deletion require accesses to this memory.

With the data of our simulation, DCF (see section II) does not experiment any overflow of counters in CBF vector. Therefore, Overflow Counter Vector are not necessary and DCF exhibits exactly the same behavior of CBF, in terms of both size and complexity.

Regarding HSBF, we have stored in scratchpad the main structure and in local memory the index tables. As said in section V, a lookup requires, for each hash function, to check the table in local memory, to search for the corresponding block in scratchpad for the bin we need and to compute a popcount. The same number of operations are required for inserting/deleting an element, with the addition of shifting by one position the bits in the bin, to increment/decrement a counter. Remember that HSBF is a simple alternative version of SBF, which is a structure optimized for multi-set. SBFs use, for values greater than 2, Elias code instead of Huffman code and several more index tables, thus resulting in higher memory consumption and operational complexity.

Table II
NUMBER OF CLOCK CYCLES FOR OPERATIONS IN THE IXP2800

Operations	Number of cycles
hash	10
popcount	1
shift	1
read/write in local memory	2
read/write in scratchpad memory	60
construction of a MPHF	1000

Finally, the overall unique structure of dlCBF has been located in scratchpad. A lookup requires $2k$ hashing and k accesses to scratchpad, while an insertion or a deletion needs $2k$ hashing, k accesses to scratchpad and, finally, k incrementing or decrementing operations.

From results in tab. III, it is clear that the solutions proposed in this paper show a significant memory saving in comparison with standard CBF and DCF (saving of 46% for ML-HCBF, 56% for ML-CCBF, and 54% for HSBF), and also compared to SBF. Instead, there is a memory consumption increase in comparison with dlCBF (from 0.93 KB up to 2.35 KB). Hovewer, our methods, inspired by dynamic approaches (e.g. DCF), avoid in a conclusive way the problem of counters overflow, thus preserving the accuracy of stored information.

Moreover, the introduction of a hierarchical structure allows in ML-CCBF a remarkable decrease of clock cycles for the lookup operation. Indeed, the main structure is stored in local memory, thus enabling lookup by accessing local memory only. The membership query is the most frequent operation for these data structures, therefore the reduction of about 83% of clock cycles for lookup is a great outcome. It outweighs the drawback of an increase of 50% of processing for inserting/deleting an element.

Finally, note that our HSBF outperforms SBF, in terms of memory consumption and operational complexity. This is an expected result, due to the simplicity of our method and to the use of Huffman code (SBFs are optimized for multi-sets). If compared to the complexity of standard algorithms, HSBF shows a reduction of 13% for lookup and an increase of 45% for insertion/deletion. The different frequency of operations allows to claim that the tradeoff is advantageous.

VIII. CONCLUSIONS

In this paper, a new upper bound for counters overflow probability in Counting Bloom Filters has been presented. This bound is very tight and has allowed the design of new high performance data structures. The target is an efficient structure

Table III
PERFORMANCE ALGORITHMS COMPARISON

	ML-HCBF	ML-CCBF	CBF	DCF	HSBF	SBF	dlCBF
Size (KB)	7.55	6.13	14.1	14.1	6.42	12.12	5.2
Main structure (KB)	7.04 (scratch.)	3.52 (local)	14.1 (scratch.)	14.1 (scratch.)	5.92 (scratch.)	8.12 (scratch.)	5.2 (scratch.)
Secondary structures (KB)	0.41 (local)	2.4 (scratch.)	-	-	-	-	-
Index tables (KB)	-	0.21 (local)	-	-	0.5 (local)	4 (local)	-
Probability of false positives	10^{-3}	10^{-3}	10^{-3}	10^{-3}	10^{-3}	10^{-3}	1.5×10^{-3}
Lookup (clock cycles)	700	120	700	700	606	801	800
Insertion/Deletion (clock cycles)	1043	1064	710	710	1058	1217	810

for systems with limited memory such as programmable routers and network processors.

Thus, three improvements to CBFs has been proposed:

- ML-HCBF, that introduces the idea of a hierarchical structure for CBF (in order to exploit the memory hierarchy of many systems);
- HSBF, that improves SBF by using Huffman code (which is optimal for independent symbols, such as the bins in CBFs);
- ML-CCBF, that combines both the advantages of the previous data structures.

A comparison among the proposed algorithms and the algorithms defined in literature has been performed by using Intel IXP2800 as referential hardware platform. The outcomes show clear memory savings of our solutions in comparison with standard CBFs (up to 50%) and a great reduction of the lookup time in ML-CCBF, even with respect to the more advanced algorithms.

We plan to use the data structures proposed in this paper in network security applications. In particular, we want to implement ML-CCBF for anti-evasion techniques, as well as intrusion detection/prevention mechanisms, on top of network processor platforms.

REFERENCES

[1] B. Bloom, "Space/time trade-offs in hash coding with allowable errors," *Communications of the ACM*, vol. 13, no. 7, pp. 422–426, July 1970.

[2] A. Broder and M. Mitzenmacher, "Network applications of bloom filters: A survey," *Internet Mathematics*, vol. 1, no. 4, 2005. [Online]. Available: http://www.internetmathematics.org/volumes/1/4/Broder.pdf

[3] L. Fan, P. Cao, J. Almeida, and A. Z. Broder, "Summary cache: a scalable wide-area web cache sharing protocol," *SIGCOMM Comput. Commun. Rev.*, vol. 28, no. 4, pp. 254–265, 1998.

[4] M. Mitzenmacher, "Compressed bloom filters," in *PODC '01: Proc. of the twentieth annual ACM symposium on Principles of distributed computing*. New York, NY, USA: ACM Press, 2001, pp. 144–150.

[5] A. Kirsch and M. Mitzenmacher, "Distance-sensitive bloom filters," in *ALENEX '06: Proc. of Algorithm Engineering and Experiments*, 2006.

[6] D. Guo, J. Wu, H. Chen, and X. Luo, "Theory and network applications of dynamic bloom filters," in *Proc. of INFOCOM 2006. 25th IEEE International Conference on Computer Communications.*, vol. 1, 2006.

[7] A. Kumar, J. J. Xu, L. Li, and J. Wang, "Space-code bloom filter for efficient traffic flow measurement," in *IMC '03: Proc. of the 3rd ACM SIGCOMM conference on Internet measurement*. New York, NY, USA: ACM Press, 2003, pp. 167–172.

[8] S. Cohen and Y. Matias, "Spectral bloom filters," in *SIGMOD '03: Proc. of the 2003 ACM SIGMOD international conference on Management of data*. New York, NY, USA: ACM Press, 2003, pp. 241–252.

[9] J. Aguilar-Saborit, P. Trancoso, V. Muntes-Mulero, and J. L. Larriba-Pey, "Dynamic count filters," *SIGMOD Rec.*, vol. 35, no. 1, pp. 26–32, 2006.

[10] F. Bonomi, M. Mitzenmacher, R. Panigrahy, S. Singh, and G. Varghese, "An improved construction for counting bloom filters," in *LNCS 4168, 14th Annual European Symposium on Algorithms*, 2006, pp. 684–695.

[11] H. Song, S. Dharmapurikar, J. Turner, and J. Lockwood, "Fast hash table lookup using extended bloom filter: an aid to network processing," in *SIGCOMM '05: Proceedings of the 2005 conference on Applications, technologies, architectures, and protocols for computer communications*. New York, NY, USA: ACM, 2005, pp. 181–192.

[12] F. C. Botelho and N. Ziviani, "External perfect hashing for very large key sets," in *16th ACM Conference on Information and Knowledge Management (CIKM07)*, November 2007.

[13] *http://www.intel.com/design/network/products/npfamily/ixp2805.htm*.

[14] *Intel® IXP2800 Hardware reference manual*.

APPENDIX
BOUND ON THE TOTAL TABLE SIZE FOR ML-CCBF

The memory size of all the tables of ML-CCBF is:

$$TS = \sum_{j=0}^{n_{tab}} TS_i \leq \left\lceil \frac{m_0}{D} \right\rceil \sum_{j=0}^{+\infty} P(\varphi \geq j) \log_2 \left[m_0 P(\varphi \geq j) \right];$$

If we define $P_0 = P(\varphi = 0)$ and $n_0 = \left\lceil \frac{m_0}{D} \right\rceil$:

$$TS \leq n_0 \left(\log_2(m_0) + (1 - P_0)[\log_2(m_0) + \log_2(1 - P_0)] \right) + n_0 \left(\sum_{j=1}^{+\infty} P(\varphi > j) \log_2 [m_0 P(\varphi > j)] \right)$$

By bound (4), the last sum is always less than:

$$\alpha^2 \sum_{j=0}^{+\infty} P(\varphi = j) \log_2 \left[m_0 \alpha^2 P(\varphi = j) \right] =$$

$$= \alpha^2 \left[\log_2(m_0) + 2 \sum_{j=0}^{+\infty} P(\varphi = j) \log_2 [\alpha P(\varphi = j)] \right] \leq$$

$$\alpha^2 \left[\log_2(m_0) + (3 - P_0) \log_2(\alpha P_0) \right]$$

which results in the following final bound (for $\alpha = \ln 2$):

$$TS \leq n_0 (2 \log_2(m_0) - 1.85)$$